

NEXORA SEARCH

Complete Project Summary & Architecture Blueprint

Software Development PSA • React + Laravel + AI + External APIs

Project identity note

The working product name is **Nexora Search**. The project should use the full name in formal materials until the final product name, domain and logo are checked for trademark/brand conflicts. This document treats Nexora Search as the academic/project label rather than claiming ownership of the broader word “Nexora”.

1. Project Vision

Nexora Search is a modern location and discovery platform built around a unified RESTful API. Instead of forcing the React client to communicate separately with many third-party services, Laravel acts as the integration layer: it receives requests, validates them, communicates with external APIs, combines and normalizes their responses, caches useful results, stores application data in MySQL, and exposes a consistent JSON API to the frontend.

Core idea: Multiple external services → Laravel integration layer → one consistent REST API → React application → optional AI-powered discovery and analysis.

2. User Experience

- **Landing page:** prominent search for cities, places and locations; popular locations; recent searches; recommendations.
- **Dashboard:** overview, recent searches, saved places, discovery activity and API insights.
- **Explore:** search and browse locations, nearby places and map results.
- **Location details:** coordinates, map, place information, weather and AI-generated summaries.
- **Saved places:** authenticated users can save and manage locations.
- **Search history:** review previous searches.
- **Admin dashboard:** users, API usage, errors, response times and system health.

3. System Architecture

React provides the client experience. Laravel provides the secure API and integration layer. External services supply maps, places, weather and other permitted data. AI services add natural-language reasoning, image analysis and intelligent summaries.

React UI → Laravel REST API → Service Layer → External APIs / AI → Cache & Queues → MySQL → JSON response → React

Controllers should remain thin. External API logic belongs in service classes; persistence belongs in the data/repository layer; API Resources define predictable JSON responses.

4. Laravel Backend

- Controllers and routes: receive and coordinate HTTP requests.
- Sanctum authentication and middleware: protect private endpoints and apply access controls/rate limits.
- Service layer: Mapbox, Google Places, GitHub, weather and AI integrations.

- Repositories/data layer: persistent database operations.
- API Resources: consistent JSON output.
- Cache, queues and jobs: reduce latency and move slow work out of the request cycle.
- Logging and monitoring: track failures, response times and provider health.
- Fallback strategy: return useful partial information when a provider becomes unavailable.

5. REST API Structure

- /api/v1/auth — registration, login and authenticated sessions
- /api/v1/search — unified location/place search
- /api/v1/locations — location information
- /api/v1/places — place data
- /api/v1/favorites — saved places
- /api/v1/history — search history
- /api/v1/weather — weather information
- /api/v1/ai — controlled AI features
- /api/v1/external — selected third-party integrations

6. Database Plan

Core tables: **users**, **locations**, **saved_places**, **search_history**, **api_requests**, **api_cache**, **failed_requests**. The **api_requests** table can record provider, endpoint, authenticated user where appropriate, response time, status code, success/failure and timestamp for observability.

7. External API Strategy

Service	Role
Google Places	Places and location information
Mapbox	Maps, geocoding and coordinates
GitHub	Developer/project information
Weather provider	Current weather and forecasts
Permitted social APIs	Selected social/page data where access permits

Dark Sky can be discussed as an API case study, but it should not be a dependency for the live system. Use an actively supported weather provider for implementation.

8. AI Layer — Puter.js

Puter.js is added as the frontend AI abstraction. Its current documentation exposes AI chat, image-to-text, text-to-image, speech features and model/provider discovery. Puter.js can accept media in AI chat, making it suitable for image analysis in Nexora Search.

Potential Nexora features:

- AI location summaries: turn raw API data into a concise human-friendly overview.
- Image analysis: users upload a photo/screenshot and ask what it contains or what information can be extracted.
- Visual place understanding: analyze images alongside location/place data.

- AI search assistant: interpret natural-language discovery requests.
- Model selector: use Puter.js model/provider discovery to expose supported models dynamically.
- Streaming responses: show AI output progressively where appropriate.

Important architecture point: Puter.js has its own provider/model system. It should not be assumed that a Puter request automatically uses the user's NVIDIA API key. Keep the Puter AI path and direct NVIDIA path conceptually separate unless a verified integration connects them.

9. NVIDIA AI Integration

NVIDIA NIM provides another AI path for Nexora Search. A key vision-capable option currently documented by NVIDIA is **nvidia/nemotron-3-nano-omni-30b-a3b-reasoning**, an omni-modal model that accepts image, video, audio and text and returns text. NVIDIA currently lists a free endpoint for it.

MiniMax M3 is also a multimodal vision-language model supporting text, image and video inputs. However, NVIDIA's current model catalog shows the hosted MiniMax M3 free endpoint with a deprecation countdown, so it should not be the sole long-term vision dependency.

Recommended AI architecture:

- **Puter.js**: flexible frontend AI experience, model discovery and user-facing AI features.
- **NVIDIA NIM**: dedicated backend AI path for controlled access to NVIDIA models, especially vision/multimodal processing.
- **Laravel**: orchestration, authentication, validation, rate limiting, logging and storage around AI requests.
- **Fallback**: allow the system to switch providers/models when a provider is unavailable or a model is retired.

10. AI + Image Analysis Flow

User selects image → React/Puter.js or Laravel upload path → validation → chosen vision model → AI analysis → normalized result → optional storage in history → display in Nexora Search.

For sensitive API keys, direct NVIDIA calls should be kept server-side rather than exposing the key in the browser.

11. Advanced Backend Features

- **Caching**: avoid repeated external API requests.
- **Rate limiting**: protect public endpoints and AI routes.
- **Queues/jobs**: move slow AI or external-provider work out of the request cycle.
- **Logging**: record provider errors, latency and important events.
- **Fallbacks**: provide partial results if a provider fails.
- **Validation**: validate inputs before calling external services.
- **API versioning**: start with /api/v1 for future compatibility.

12. Admin & Monitoring

The admin area turns the project into a small platform rather than a simple API consumer. It can show total requests, successful requests, failures, average response time, most searched locations, provider usage, AI requests and recent errors.

13. Visual Identity

Logo direction: an abstract **N** combined with a location/navigation and connection motif. The chosen symbol does not need to be a literal compass; its stronger meaning is discovery, connected information and technology.

Design direction: clean modern SaaS. Use light/white surfaces, deep navy text, a controlled blue accent, subtle borders, cards and restrained motion. Avoid an overly neon/gaming aesthetic.

14. Technology Stack

Area	Technology
Frontend	React
Backend	Laravel / PHP
Database	MySQL
API format	JSON / REST
Authentication	Laravel Sanctum
AI frontend	Puter.js
AI / vision	NVIDIA NIM + selected multimodal models
API testing	Postman
Source control	Git + GitHub
Performance	Laravel Cache + Queues

15. PSA Positioning

The strongest academic framing is not “a website that searches places.” Nexora Search is a **unified RESTful API platform that aggregates heterogeneous third-party services, normalizes their data, applies authentication and performance controls, integrates AI capabilities, persists useful application data, and exposes a clean JSON interface to client applications.**

- Research REST principles, HTTP methods, API architecture and operation.
- Design the conceptual framework and system architecture.
- Compare and practically interact with at least five APIs.
- Document authentication, data formats, operations, limitations and use cases.
- Study Laravel HTTP clients, JSON, MySQL, authentication, caching, queues and testing.
- Implement the RESTful service with Laravel/MySQL and React as the client.
- Demonstrate how a unified backend improves consistency, performance and usability.

16. Suggested Presentation Structure

1. Title — Nexora Search
2. Problem / Motivation
3. Aim & Objectives
4. What is REST?
5. REST principles & HTTP methods
6. API comparison — five or more APIs
7. Proposed system architecture
8. Laravel backend architecture
9. Database design
10. React frontend design
11. External API integration flow
12. AI architecture: Puter.js + NVIDIA
13. Authentication, security & rate limiting
14. Caching, queues & error handling
15. Postman testing / sample JSON
16. Admin monitoring dashboard
17. Final architecture and advantages
18. Limitations / future improvements
19. Conclusion

17. Build Order

Phase 1: Finalize name, logo and visual design.

Phase 2: Design database schema and API contracts.

Phase 3: Create Laravel structure and authentication.

Phase 4: Implement one external API end-to-end.

Phase 5: Add additional integrations and response normalization.

Phase 6: Add caching, rate limiting, logging and queues.

Phase 7: Add Puter.js AI features.

Phase 8: Add controlled NVIDIA multimodal/vision integration.

Phase 9: Connect React and build dashboard/admin screens.

Phase 10: Test with Postman and document screenshots/results.

Phase 11: Prepare PowerPoint and live demonstration.

18. Final Product Goal

Nexora Search should feel like a real software platform: polished React interface, structured Laravel API, multiple external integrations, secure authentication, caching and observability, plus an AI layer capable of intelligent text and image understanding. The project remains academically grounded in RESTful web services while demonstrating modern full-stack and AI integration skills.

Reference notes

Current Puter.js documentation supports AI chat with media inputs and model/provider discovery. NVIDIA currently documents free hosted endpoints for multimodal models including Nemotron 3 Nano Omni; MiniMax M3 is multimodal but currently shown with a hosted-endpoint deprecation countdown. Verify model availability, quotas, terms and endpoint status again when implementation begins.